

CSCE 611 OPERATING SYSTEMS · MODULE 13 · LECTURE NOTES

Kernel Design

Monolithic, micro, and what the argument was really about.

MODULE OUTCOMES

Compare monolithic and microkernel designs on concrete criteria.

Explain why IPC cost decided the historical argument.

Explain formal verification and what seL4 achieves.

Explain modern hybrids: modules, eBPF, unikernels.

Judge kernel design choices against workload requirements.

1 The question

Every operating system must decide which code runs with full privilege. That code is the **trusted computing base**: a bug anywhere in it can compromise everything.

	Monolithic	Microkernel
In kernel mode	Scheduling, memory, file systems, device drivers, network stack, and more.	Only IPC, scheduling, and address space management.
Everything else	—	Ordinary user processes: drivers, file systems, network stack.
A service request is	A function call.	An IPC message to another process.
Performance	Fast.	Depends entirely on IPC cost — see §2.
A driver bug	Can corrupt any kernel data structure, including another subsystem's.	Kills one user process; the system continues.
Trusted computing base	Millions of lines.	~10,000 lines.
Examples	Linux, Windows NT, FreeBSD, xv6.	seL4, QNX, Minix 3, L4 family.

2 Why monolithic kernels won the desktop

THE ARGUMENT WAS DECIDED BY IPC COST

In a microkernel, every file read becomes at least one IPC round trip to the file system process, which may itself IPC to the driver process. A function call becomes several context switches.

First-generation microkernels — Mach above all — had IPC costs in the thousands of cycles. The resulting performance penalty was large enough to decide the matter in practice, and it is why the Tanenbaum–Torvalds debate of 1992 was settled by the market rather than by argument.

Jochen Liedtke demonstrated this was an implementation failure, not a structural one. By designing L4 around IPC from the beginning — minimal message formats, careful register use, no unnecessary copying — he reduced the cost by roughly an order of magnitude, to a few hundred cycles.

By that point Linux and Windows NT had established themselves on other grounds entirely: hardware support, developer familiarity, and momentum. The microkernel case was correct and arrived too late, which is a recurring pattern in systems research.

3 Where microkernels did win

System	Deployment	Why it won there
QNX	Automotive infotainment and control, medical devices, industrial automation.	Fault isolation and hard real-time guarantees. A failing component must not take the system down, and certification requires being able to argue that it cannot.
seL4	Defence, avionics, high-assurance systems.	Formal verification. See below.
L4 family	Billions of mobile phone basebands.	The baseband processor runs proprietary firmware that must be isolated from the application processor. This is quietly the largest microkernel deployment in existence.
Zircon (Fuchsia)	Google devices.	Driver isolation and independently updatable drivers — a direct response to Android's difficulty in shipping kernel updates across vendors.

SEL4 AND WHAT VERIFICATION ACTUALLY MEANS

seL4 carries a machine-checked proof that its C implementation satisfies its formal specification, together with proofs of integrity and confidentiality enforcement.

This is categorically stronger than extensive testing. There are no buffer overflows, no null pointer dereferences, no undefined behaviour — not 'none have been found', but *none exist*, demonstrated mathematically.

It is achievable because the kernel is roughly 10,000 lines. The proof is around 200,000 lines of Isabelle/HOL and took person-decades. **That ratio is precisely why nobody has verified Linux**, and it will not change.

The lesson generalises: a small trusted computing base is not merely easier to audit, it is the precondition for proving anything about the system at all. Architecture determines what assurance is even possible.

4 What actually happened: convergence

The interesting outcome is that monolithic kernels adopted most of the microkernel programme without adopting the architecture.

Mechanism	What it provides	The microkernel idea it adopts
Loadable modules	Drivers and file systems loaded and unloaded at runtime rather than compiled in.	Modularity — though still in kernel mode, so without the isolation.
FUSE	File systems implemented as ordinary user processes.	Services outside the kernel. sshfs, s3fs, and many others rely on this.
User-space drivers	USB, GPU, and high-performance networking drivers substantially in user space.	Driver isolation. DPDK and SPDK bypass the kernel entirely for performance, which is the microkernel argument made for speed rather than safety.
eBPF	Sandboxed, verified programs that extend kernel behaviour.	Safe extension — see below.
Hypervisors	A small, carefully audited layer beneath the kernel.	A minimal trusted computing base. Arguably the microkernel idea succeeding under a different name.

EBPF IS A GENUINELY DIFFERENT ANSWER

The microkernel question was: *should extensions run in a separate address space?* eBPF answers a different question: *can we run them in the kernel but prove in advance that they are safe?*

Before an eBPF program is loaded, a verifier establishes that it terminates (no unbounded loops), that every memory access is within permitted bounds, that types are respected, and that it cannot crash the kernel. Only then is it accepted.

The outcome is kernel-speed extension with no process boundary, no IPC, and no trust placed in the program's author. It is used for packet filtering and routing, tracing and observability, security policy enforcement, and increasingly for scheduling.

This is neither monolithic nor microkernel: it replaces a *runtime* isolation boundary with a *static* guarantee. After forty years of the same argument, that is a genuinely new position.

4.1 Unikernels: the opposite direction

A **unikernel** compiles the application together with just the operating system functionality it needs into a single binary running in one address space, usually directly on a hypervisor.

There is no kernel/user boundary and no system call mechanism — a 'system call' is a function call. The reasoning is that in a virtual machine dedicated to one application, the kernel's protection and multiplexing are redundant: the hypervisor already provides isolation, and there is no second program to be protected from.

The result boots in milliseconds, occupies a few megabytes, and presents a minimal attack surface. The costs are real: debugging is difficult, there is no process model, and the tooling is immature. It is a useful reminder that 'what is a kernel for' has a different answer when it serves exactly one program.

5 Judging a design

Requirement	Appropriate design	Reason
General-purpose computing, broad hardware support.	Monolithic with loadable modules.	Performance and the enormous driver ecosystem. Linux and Windows.
Safety certification or formal assurance.	Microkernel — seL4.	A small TCB is the precondition for proof.
Hard real-time with fault isolation.	Microkernel — QNX.	Bounded latency and the ability to argue that a component failure is contained.
Multi-tenant cloud running untrusted code.	Hypervisor with lightweight VMs.	A small, hardware-assisted isolation boundary (Module 11).
One application per virtual machine.	Unikernel or library OS.	The protection the kernel provides is redundant.

WHERE THIS COURSE LEAVES YOU

You have written a context switch in assembly, replaced a scheduler, implemented demand paging and copy-on-write, added system calls, and made a file system survive crashes. Those are the things an operating system is.

More usefully, you know what the illusions cost and where each one leaks: that a write is not durable until fsync, that a context switch costs more in cold cache than in registers, that a container is a process with flags, and that no timeout distinguishes slow from dead.

CSCE 614 was the hardware beneath all of it. **CSCE 662** and **678** take Module 12 seriously and build real distributed systems. **CSCE 713** takes Module 11 seriously. **CSCE 735** takes the concurrency of Modules 04 and 05 to many cores.

And you can now read kernel source and know what you are looking at, which is the thing that actually transfers.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Klein et al. — seL4: Formal Verification of an OS Kernel (free)	What was proved, how, and what it cost. The most significant result in kernel design in decades.
Liedtke — On micro-kernel construction (free)	The paper that showed microkernel IPC could be fast, and why the first generation was not.
The Tanenbaum–Torvalds debate (1992, archived)	The original argument, in full. Worth reading for how both sides were partly right.
Brendan Gregg — eBPF resources	What eBPF can actually do, from someone who uses it daily.
Madhavapeddy et al. — Unikernels: Library Operating Systems for the Cloud (free)	The case for collapsing the boundary entirely.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure xv6's system call overhead: time a null system call and compare against a null function call. Compute the ratio.
- 2 Implement a trivial FUSE file system in user space. Measure its throughput against the equivalent kernel file system and attribute the difference.
- 3 Write and load an eBPF program that counts system calls by number. Note what the verifier rejects before it accepts your program.
- 4 Count the lines of code in xv6 by subsystem, then in seL4, then in the Linux kernel. Plot trusted computing base size against the three designs and consider what is being traded.
- 5 Read the Tanenbaum–Torvalds debate and write a one-page assessment of which predictions held, now that you have built a kernel.
- 6 Take one subsystem you implemented in this course and sketch how it would be structured in a microkernel. Identify every place a function call becomes IPC and estimate the cost.
- 7 **Project 2 is now due.** Complete the virtual memory and crash-consistent file system work described in the syllabus, including the hundred-run crash injection results.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What runs in kernel mode in each design, and what does a service request cost in each?
- 2 Why did monolithic kernels win, and in what sense was the microkernel argument correct?

- 3 Name three places microkernels are deployed in production and why each chose one.
- 4 What does seL4's verification establish, and why is it only possible for a small kernel?
- 5 Name four microkernel ideas that monolithic kernels adopted.
- 6 How does eBPF differ from both designs, and what does it replace a runtime boundary with?
- 7 When would a unikernel be the right choice, and what is given up?